

Kaveri Architecture

1. About Kaveri

Kaveri is a digital platform designed to support end-to-end transactions of the registration process, involving both citizen-facing and internal departmental workflows. It serves as the primary interface for users through a web-based UI and coordinates multiple backend capabilities such as identity management, reporting, and integrations with external government and third-party systems.

The system has been live in production for over two years and currently operates with minimal manual intervention. The primary architectural objective during the current phase has been **stability, operational predictability, and maintainability**, rather than large-scale redesign.

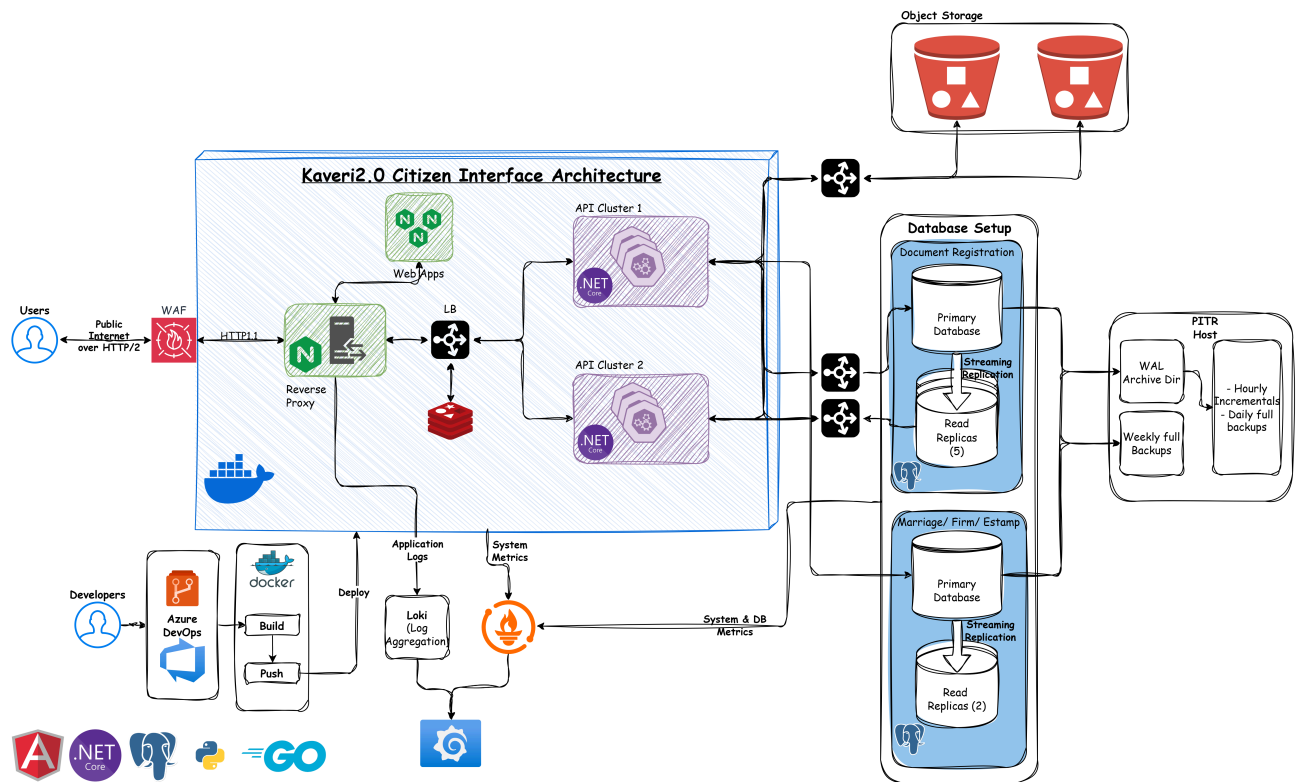
2. Architecture Style and Principles

The system follows a **service-oriented architecture** with the following characteristics:

- Multiple independently deployed backend services implemented using .NET Core (dotnet 8), python, golang.
- A single-page web application built with Angular (v13) acting as the primary user interface.
- A shared PostgreSQL database instance used by multiple services.
- Synchronous HTTP-based communication as the dominant interaction pattern.
- Limited use of asynchronous processing for background and batch workloads.

Kaveri has two interfaces, citizen and department facing interfaces. The architecture and deployments of both the interfaces are quite similar, with the only deviation being that the department interface is exposed only KSWAN network and with some additional business logic which is relevant only for the departmental workflows and should not be exposed to the citizen interface.

While the system is often referred to as “*microservices-based*”, it unintentionally retains certain centralized elements (notably the database) due to operational, historical, and reporting constraints. Below is a high level overview of the citizen facing system,



Primary entry points into the system include:

- Web UI accessed by external users and internal operators.
- Internal service-to-service APIs.
- Scheduled and batch jobs for reconciliation, reporting, and maintenance.
- External systems integrating via exposed APIs or secure data exchange mechanisms.

PS: The setup was serving application and accepting connection over HTTP/2 protocol, this has now been affected with the introduction of WAF and only HTTP1.1 is being served. Resolution has to be sought from SDC along with the WAF vendor.

3. Core Components

At a high level, the system consists of:

- **Reverse Proxy**
 - Single point of entry into the system; all traffic must go through this layer.
 - Provides following functionalities
 1. SSL Offloading.
 2. Rate limiting.
 3. Micro caching of requests.
 4. Blocking major crawlers, bots and API Clients.
 5. Load balancing across UI instances and the API gateway instances.
- **API Gateway**

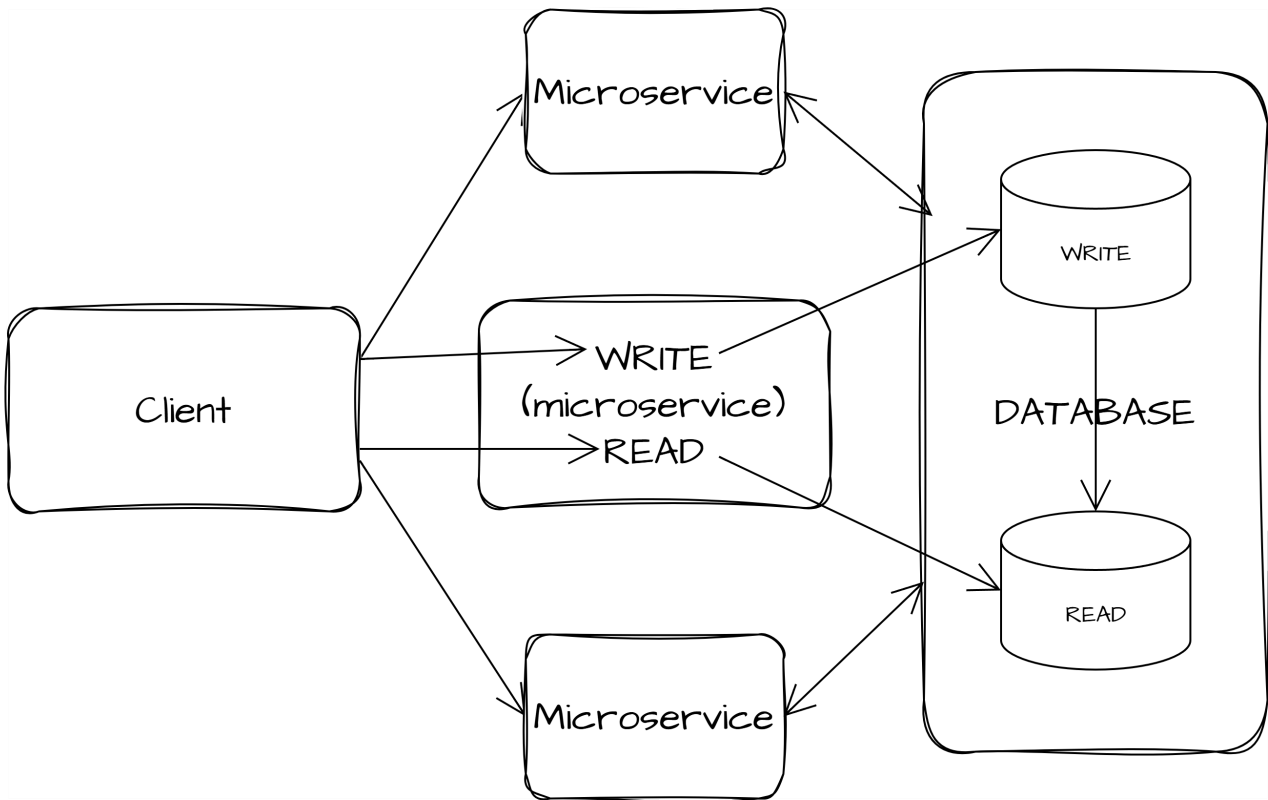
- API gateway manages the traffic across all the services and is responsible for relaying/ routing the response to correct requester.
- Offers the following functionalities
 1. Rate limits at IP and route level. (Unless explicitly allowed.)
 2. Load balancing across multiple instances of services.
 3. Validations for User authentication and authorization for every incoming request. (Unless explicitly allowed.)
 4. Session management through the use of a `redis` cache.
 5. Request signature validation. (WIP; To be moved to production)
- **UI Layer**
 - Angular-based single-page application.
 - Handles user interaction, client-side validation, and orchestration of backend calls.
 - Closely coupled to backend API contracts.
- **Service Layer**
 - Services responsible for business logic and workflow enforcement.
 - APIs exposed for UI and internal and/ or external consumption.
 - Data access implemented using `Npgsql` with `Dapper` .
 - Business rules enforced at the service level.
 - While most business logic is in `dotnet` , some functionalities are in `python` and `golang` .
- **Data Layer**
 - PostgreSQL database shared across services.
 - Tables logically grouped by domain i.e. document registration, marriage registration, e-Stamp etc.
 - Indexing and performance tuning applied based on production usage patterns.
- **Cache Layer**
 - Redis instance for session and user management.
 - Response caching at both proxy and gateway level for quicker response without compromise on quality of data.
- **Integrations**
 - External systems accessed via synchronous APIs or scheduled jobs.
 - Integration logic encapsulated within dedicated service components where possible.

3.2 Database

The PostgreSQL database is shared across services. To mitigate the risks typically associated with shared databases, the following principles are applied:

- Data ownership is at the application level.

- There is no logical data ownership defined per service.
- Schema evolution and write operations can be through any service
- Non-owning services are expected to use read-only access or service APIs.
- Follows a Replica pattern, with read replicas present to support with the offloading of read operations.



3.3 Integrations

The external integrations with Kaveri and their respective purposes are as follows:

- **Bhoomi & Mojini:**
 1. Pulling of agricultural properties.
 2. Pushing mutation details of the so registered properties.
- **eSwathu/ RDPR**
 1. Pulling of non-agricultural rural properties.
 2. Pushing mutation details of the so registered properties.
 3. Pushing of document details, encumbrance certificates and signed documents on a need basis for eKhata.
- **eAasthi & BBMP**
 1. Pulling of urban properties.
 2. Pushing mutation details of the so registered properties.
 3. Pushing of document details, encumbrance certificates and signed documents on a need basis for eKhata.

- **ULMS**
 1. Pulling of non-eAasthi urban properties
 2. Pushing mutation details of the so registered properties.
- **FRUITS/ eSaala**
 1. Automatic filing/ registration of mortgages or release against agricultural properties.
 2. Updating the closure of such transactions.
 3. Pushing of 1-day ECs on a need basis.
- **Kaveri 1**
 1. EC and CC services. Pulling of EC and CC for Kaveri 1 registered documents are through this service.
 2. Services to fetch pending and completed registration data.
- **Khajane Payment Gateway**
- **eKYC & eSign Services**
- **Sakala**, to ensure timely delivery of services.
- **Scality Service**, for the upload/ download of all documents.
- **SMS Service**, for the SMS based notifications.
- **Mail Service**, for the mail based notifications.
- **KGIS**, for the map layer embedded in Kaveri.
- **NGDRS**, to push user details on a need basis.

4. Operations

Key operational characteristics of the system include:

- Moderate-high concurrency during peak usage windows, close to ~4.5-5k concurrent users at any given point from 1000h-1800h.
(Expected to increase with eStamp module being made live.)
- Latency-sensitive user-facing workflows.
- Tolerance for eventual consistency in non-critical processes.
- Observability present across services and environments.
- Below table gives the scale of the transactions in Kaveri over the last financial year (FY 2025-26).

Service	Max	Avg
Document Registration	~17k	~8k
Encumbrance Certificate	~30k	~14k
Certified Copy	~11.5k	~5k

5. Deployment and Environments

5.1 Deployments

Each of the services, web application, DB migrations is a unit of deployment, which may or may not be deployed independently depending on the nature of the change. Standard deployment flow is as follows:

- Build package, ensure no build errors.
- Push the artifact.
- Deploy to target environment.
- Post deployment checks
 - Ensure service is running.
 - DB connectivity is unbroken.
 - No breaking downstream dependencies.
(Let's say, API gateway with its cache layer dependencies, for example.)

5.2 Environments

There are three environments, primarily

- Development
 - Hosted on azure cloud.
 - Used for active development.
 - Data maybe synthetic or fabricated.
 - Deployments are automated through Azure Pipelines .
- Staging
 - Hosted on SDC.
 - Mirrors production as closely as possible.
 - Data/ schema changes to be validated here first.
 - Requires constant housekeeping/ maintenance to ensure sync with production.
 - Deployments are through one of these mechanisms: deployment scripts, Jenkins, or portainer .
- Production
 - Hosted on SDC.
 - Changes in this environment directly is not advisable.
 - Emphasis on minimal downtime and on not affecting the operations adversely.
 - Deployments are right now through deployment scripts.
The portainer and docker swarm setup has to be promoted to Production.
Promotion flow: Development -> Staging -> Production

The endpoints of VMs/ servers in the production cluster are as listed below

	Servers	Purpose	
Application Servers	172.18.35.142	Citizen	Interface
	172.18.35.150		API Cluster 1
	172.18.35.151		API Cluster 2
	172.18.35.143	Dept	Interface and APIs
DB Servers	172.31.58.214	Document Registration	Primary DB
	172.31.58.215		Standby 1
	172.31.26.39		Standby 2
	172.31.26.40		Standby 3
	172.31.26.41		Standby 4
	172.31.58.206		Standby 5
	172.31.58.218		Standby 6
	172.31.58.219		Standby 7
	172.31.58.216	Marriage/ Firm/ Estamp	Primary DB
	172.31.58.211		Standby 1
	172.31.58.212		Standby 2
Misc	172.18.35.155	Maintenance Server	
	172.31.26.44	DB Maintenance/ backup server	
	172.31.55.243	Scality Server	

5.3 Rollback strategy

- Artifacts from the stable/ current to be retained to ensure operations.
- Rollback scripts for the database to be always maintained.
- RCA.

6. Current Architectural Focus

The current architectural focus has been on:

- Establishing service boundaries, data ownership, and making contracts explicit.
- Reducing accidental coupling and undocumented dependencies.
- Internal refactoring without destabilizing/ compromising on the functional behavior to,
 1. Improve maintainability.
 2. UI Redesign with UX, security and data considerations.
 3. Harden the UI layer with robust client-side validations and migrate the business logic to the service layer.
- Documenting architectural and process decisions to support long-term handover.

7. Architectural Constraints

The following constraints materially influence architectural decisions:

- The system must operate within state infrastructure (SDC) and compliance requirements.
- Database separation is constrained by dependencies and operational risk.
- Downtime tolerance is low for user-facing workflows.
- Architectural changes must prioritize backward compatibility and rollback capability.

These constraints are considered non-negotiable unless explicitly revisited and approved.

8. Known limitations

8.1 Shared Database Across Services

Multiple services access the same database instance, this is owing to the historical implementation decisions. This can be mitigated, to a certain degree, by defining logical boundaries for the data ownership.

8.2 Legacy Database Schema

Some domain logic including the database schema have been carried forward from Kaveri 1. This is owing to early delivery constraints, backward compatibility requirements and evolving understanding of the domain. Any change to these must account for all the dependencies and accompanied by a thorough impact analysis.

8.3 Synchronous Service-to-Service Calls

Most inter-service communication is synchronous in nature, as it is simpler to manage. The async orchestration might hurt latency sensitive workflows in the application. Latency analysis is a must when further nesting service calls inside these services.